

Testing Throughout the Software Development Lifecycle



Chapter 2 learning route

Syllabus numbering retained: 2.1 → 2.3

2



2.1
Testing in SDLC
context

2.1.1 Impact of the
Software
Development
Lifecycle on Testing
2.1.2. Software
Development
Lifecycle and Good
Testing Practices
2.1.3
Test-first approaches

2.1.4 DevOps and
testing
2.1.5 Shift-left
approach
2.1.6
Retrospectives
and process
improvement

2.2
Test levels and types

2.3
Maintenance testing



INTERACTIVE: Ask learners which SDLC model they have experienced and how testing was organized.

2.1 Testing in the Context of an SDLC

The lifecycle model changes how, when, and by whom testing is performed.



SDLC model

- Abstract view, high level representation of software development process
- Defines both logical and chronological relation between activities performed and development process.
- Examples: waterfall, V-model, spiral, prototyping, Unified Process



Detailed software development methods and practices

- Scrum, Kanban, Lean IT, XP
- Test-Driven development (TDD), Acceptance test-driven development (ATDD), Behavior-driven Development (BDD)
- Domain-driven design (DDD) and other methods can influence testing.



Testing impact

- Scope and timing of activities
- Documentation detail
- Techniques, automation and tester role and responsibilities



INTERACTIVE: Compare waterfall vs agile: what changes for a tester?

2.1.1 Impact of the SDLC on testing

Testing must be adapted to the selected lifecycle.



Sequential models

- Testers review requirements and design early, participate in test analysis.
- Dynamic testing usually waits until executable code exists, cannot be performed early phase.
- Documentation tends to be more formal



Iterative, incremental and agile models

- Each iteration may include static and dynamic testing performed at all test levels
- Frequent increments require fast feedback and regression testing
- Regression testing and automation become more important
- Agile software development assumes that change may occur throughout the project where, lightweight work product documentation and extensive test automation involve in agile projects.



INTERACTIVE: Ask learners to identify one advantage and one testing risk in each model.

2.1.2 Good testing practices across all SDLCs

Certain principles remain valid regardless of the lifecycle.

2



Good testing practices of the chosen SDLC model, include the following:

- For every software development activity, there is a corresponding test activity means all development activities are subject to quality control.
- Different test levels have specific and different test objectives, which allows for avoiding redundancy.
- Test analysis and design for a given test level begins during the corresponding development phase of the SDLC, so that testing can reflect to the principle of early testing.
- Testers are involved in reviewing work products as soon as drafts of this documentation are available, so that this earlier testing and defect detection can support the shift-left strategy.



INTERACTIVE: Ask: which practice best supports early testing and why?



2.1.3 Testing as a driver for development

TDD, ATDD and BDD define tests before implementation.



Test-Driven Development (TDD)

- Developer-level test-first approach
- Write test → write code → refactor
- Tests direct coding at component level



Acceptance Test-Driven Development (ATDD)

- Acceptance tests derive from acceptance criteria
- Business, developers and testers collaborate
- Tests guide implementation of a user story



Behavior-Driven Development (BDD)

- Desired behavior written in simple language
- Often uses Given / When / Then

Given: the initial context at the beginning of the scenario, in one or more clauses;

When: the event that triggers the scenario;

Then: the expected outcome, in one or more clauses.

- Scenarios can become executable tests



2.1.4 DevOps and testing

DevOps approach integrates development, testing and operations, this will enable the teams to build, test and release high-quality code through a delivery pipeline.



Testing benefits

- Fast feedback on code quality
- Stable environments via continuous integration (CI)/ continuous delivery (CD)
- Reduced repetitive manual testing
- Broader view of non-functional quality



Risks and challenges

- Pipeline must be defined and maintained
- Automation needs resources and skills
- Manual user-perspective testing is still needed



2.1.5 Shift-left approach

Move testing and quality activities earlier—but do not neglect later testing.



Ways to shift left

- Review specifications from the testing perspective where might find potential defects.
- Write test cases before code is written and run where appropriate during code implementation.
- Use CI/CD for fast feedback
- Run static analysis of source code before dynamic testing as part of an automated process.
- Start selected non-functional testing earlier when possible as at the component test level when a complete system and test environment is ready.



2.1.6 Retrospectives and process improvement

Retrospectives turn experience into improvement actions, and also known as “post-project meetings” and project retrospectives. This situation are often held at the end of a project or an iteration, at a release milestone, or can be held when needed. In these retrospectives meetings involve the participants not only testers, but also e.g., developers, architects, product owner, business analysts

Ask three questions

- What worked well?
- What did not work?
- How do we improve and retain success?

Testing benefits

- Improved test effectiveness and efficiency
- Better testware quality
- Better cooperation between development and testing

Good practice

- Record results
- Follow up actions
- Choose a small number of improvements at a time



2.2 Test levels and test types

Levels describe where in development testing occurs; types describe what quality objective is targeted.

Test level

- A group of test activities managed together
- Performed at a given stage of development
- Example: component, component integration, system, system integration, acceptance

Test type

- A group of activities focused on a characteristic
- Can be performed at any level
- Example: functional or non-functional testing
- **Functional testing** evaluates the functions that a component or system should perform.
- **Non-functional testing** evaluates attributes other than functional characteristics of a component or system.
- **Black-box testing** is specification-based and derives tests from documentation external to the test object.
- **White-box testing** is structure-based and derives tests from the system's implementation or internal structure (e.g., code, architecture, work flows, and data flows).

Key distinction

- Level = stage/object scope
- Type = test objective / quality focus
- They combine in practice



2.2.1 Test levels overview

Five levels commonly described in Foundation Level.



Component

single module / unit

Component integration

interfaces between
components

System

whole system behavior

System integration

interfaces with other systems

Acceptance

readiness and user needs



INTERACTIVE: Ask learners to place their latest test activity into one of these levels.

2.2.1 Component and component integration testing

Lower levels focus on isolated components and their interactions.

2



Component testing

- Tests individual components in isolation
- Often uses stubs, drivers, mocks or unit frameworks
- Usually performed by developers



Component integration testing

- Tests interfaces and interactions between components
- Depends on the integration strategy
- Often automated and linked with CI



2.2.1 System and system integration testing

Higher technical levels focus on complete behavior and external interfaces.

2



System testing

- Tests the behavior and capabilities of the entire system
- Includes functional and non-functional behavior
- Often performed by independent testers



System integration testing

- Tests interfaces with other systems and services
- Needs an environment close to operational reality
- Focuses on data exchange, protocols and external dependencies





2.2.1 Acceptance testing

Acceptance testing focuses on validation and readiness for deployment.



UAT

- Business users validate fitness for use
- Focus on business processes and user needs



Operational acceptance

- Operators check deployment, backup, recovery, user management, security and operations



Other forms

- Contractual / regulatory acceptance
- Alpha testing at producer site
- Beta testing in customer environments



INTERACTIVE: Ask: why is finding many serious defects during UAT a project risk?

2.2.2 Test types overview

Four test types in the syllabus can be applied at different test levels.



Functional

- Tests what the system should do
- Focus: completeness, correctness and appropriateness



Non-functional

- Tests how well the system behaves
- Examples: compatibility, maintainability, portability, performance, usability, security, reliability



Black-box / white-box

- Black-box: specification-based behavior
- White-box: internal structure and processing



INTERACTIVE: Ask learners to classify examples as functional or non-functional.

2.2.2 Functional vs non-functional testing

What the system does vs how well it behaves.



Functional testing

- Evaluates functions described in requirements, stories or use cases
- Often uses black-box techniques
- Coverage can be traced to functional requirements



Non-functional testing

- Evaluates quality characteristics such as performance, compatibility, usability, reliability, security, maintainability and portability
- Should start early where possible
- Often needs specialist skills or environment



INTERACTIVE: Write one functional and one non-functional test objective for an online booking system.

2.2.2 Black-box vs white-box testing

Behavior-based testing and structure-based testing complement each other.



Black-box testing

- Derived from external documentation or specification
- Independent of implementation details
- Useful even if implementation changes but behavior remains the same



White-box testing

- Derived from internal structure such as code, workflows or architecture
- Measures coverage of structure
- Depends on knowledge of design or implementation



2.2.3 Confirmation testing and regression testing

Both respond to change, but they answer different questions.

2



Confirmation testing

- Checks whether the original defect has been successfully fixed
- Often reruns the failed test and may add focused tests
- Question: “Is this fix correct?”



Regression testing

- Checks whether change has caused adverse side effects
- May affect same component, other components, connected systems, or environment
- Question: “Did we break anything else?”



2.3 Maintenance testing

Maintenance testing evaluates changes to an operational system and impact on unchanged areas.



Scope depends on

- Risk of the change
- Size of existing system
- Size of the change



Testing focus

- Success of implementation
- Regression in unchanged areas
- Impact analysis before or after change



Operational reality

- Can involve planned releases or hot fixes
- Can be corrective, adaptive or improvement-related



INTERACTIVE: Ask learners to propose tests for a payment gateway upgrade.

2.3 Triggers for maintenance testing

Maintenance testing is triggered by more than defect fixes.



Modifications

- planned enhancements
- corrective changes
- hot fixes



Upgrades / migrations

- platform changes
- environment changes
- data conversion between applications



Retirement

- data archiving
- restore and retrieval procedures
- end-of-life transition checks



Chapter 2 recap and checkpoint

Check the main distinctions before moving to Chapter 3.

2



Checkpoint questions

- How does the SDLC model influence testing?
- What is the difference between test-first, DevOps and shift-left?
- How do test levels differ from test types?
- What is the difference between confirmation and regression testing?
- What are common triggers for maintenance testing?

